REVIEW 4

Container Management Shell

25CS2104E — Operating Systems and Systems Programming

Instructor: Dr. P. Kalpana

Vidya Keerthi Chitturi — 2510030372

Badam Hasika — 2510030403

Shaik Asha Parveen — 2510030225

Madi Amulya — 2510030158

Made with GAMMA

Problem Statement and Objectives

Problem	Objective
Understand operating-system command execution.	Develop a modular shell in C on Ubuntu.
Manage processes and external commands.	Demonstrate POSIX process-management APIs.
Connect commands through pipes.	Implement built-in commands, pipelines, and redirection.
Redirect input and output.	Test with Valgrind, GDB, and AddressSanitizer.
Validate memory and runtime behavior.	Educational command-line shell; not a full container runtime.

Our project's novelty is the integration of core operating-system mechanisms into a single modular shell. Instead of only using existing Linux commands, we implemented command parsing, process management, inter-process communication, I/O redirection, and background monitoring ourselves. This makes it a practical learning platform for understanding how a shell works internally.

Made with GAMMA

Architecture and Technical Stack

User Input → Dynamic Input Handling → Command Parsing → Built-in or External Command Execution → Pipes/Redirection → Output

Supporting modules

- Signal handling
- Detached monitoring thread
- Dynamic memory management

First, the user enters a command. The shell reads the input dynamically and parses it into a command and its arguments. It then checks whether the command is a built-in command or an external program.

External commands are executed using process-management functions. Pipes and redirection handle communication and file input/output.

Technology/API and Purpose

Technology/API	Purpose
C and Linux/Ubuntu	Implementation environment
`malloc()`, `realloc()`, `free()`	Dynamic input memory
`fork()`, `execvp()`, `waitpid()`	Process execution
`pipe()`, `dup2()`	Two-command pipelines
`open()`, `close()`, `dup2()`	File redirection
POSIX threads	Periodic monitoring

Made with GAMMA

Implemented Features

Feature	Status
Interactive shell prompt and dynamic input	Implemented
Command parsing	Implemented
External command execution	Implemented
Built-ins: `cd`, `pwd`, `clear`, `help`, `env`, `exit`	Implemented
Signal handling for `SIGINT` and `SIGCHLD`	Implemented
Two-command pipelines	Implemented
Input, output, append, and standard-error redirection	Implemented
Detached monitoring thread	Implemented
Memory debugging and validation	Tested

Scope note: no claim of arbitrary-length pipelines or actual container lifecycle management.

Made with GAMMA

Accuracy of Results

Test	Command	Observed Result
Built-in help	`help`	Built-in command list displayed
Working directory	`pwd`	`/home/vidya/OSSP_Container-management-shell`
Directory listing	`ls`	Project files and directories displayed
Pipeline	`ls src | grep c`	Matching C source filenames displayed
Output redirection	`echo Hello > output.txt`	Output redirected to a file
File read	`cat output.txt`	Displayed `Hello`
Input redirection	`wc -c < output.txt`	Returned `6`

These results demonstrate corresponding features under tested conditions, not exhaustive correctness.

For redirection, we used `echo Hello > output.txt`, verified the contents using `cat output.txt`, and tested input redirection using `wc -c < output.txt`. It returned six bytes, as expected for the word Hello followed by a newline.

Made with GAMMA

Technical Competence

Concept	API	Role
Process creation	`fork()`	Creates a child process for command work.
Program execution	`execvp()`	Replaces the child process with an external program.
Process waiting	`waitpid()`	Waits for a child process and collects its status.
Inter-process communication	`pipe()`	Connects output from one command to another.
File descriptor duplication	`dup2()`	Routes standard input or output to a descriptor.
File operations	`open()`, `close()`	Opens and closes files for redirection.
Signal handling	`SIGINT`, `SIGCHLD`	Responds to interrupts and child-process state.
Monitoring	`pthread_create()`, `pthread_detach()`	Starts and detaches the monitoring thread.

Made with GAMMA

Memory Validation and Debugging

Valgrind — observed result

- Total heap usage: 8 allocations, 7 frees
- Definitely lost: 0 bytes
- Possibly lost: 272 bytes in 1 block
- Error summary: 1 error context
- Stack trace points to thread creation and thread-local storage allocation

AddressSanitizer — observed result

- `make asan` completed without compiler errors
- Shell ran `help`, `pwd`, and `exit`
- No AddressSanitizer errors appeared during that test

Observed results are not proof that every possible memory issue is absent.

We used Valgrind and AddressSanitizer to examine memory-related behavior.

In our recorded Valgrind test, there were eight allocations and seven frees. Valgrind reported zero bytes definitely lost, but it also reported 272 bytes as possibly lost in one memory record. The error summary showed one error context associated with thread creation and thread-local storage allocation.

We did not modify the provided thread implementation. We have recorded this warning as an area for further investigation.

We also built the program using AddressSanitizer and ran the shell through the help, current-directory, and exit commands. No AddressSanitizer errors appeared during that test.

Made with GAMMA

Innovation and Sustainability

Feature	Value
Modular source code	Easier maintenance
Dynamic input handling	Flexible command input
Pipes and redirection	Command composition and controlled I/O
POSIX APIs	Practical systems-programming learning
Monitoring thread	Periodic status output
Debugging tools	Evidence-based validation

Sustainability focus: reusable modules, low-resource command-line operation, maintainability, and extensibility. No measured environmental benefits are claimed.

The main value of our project is its modular implementation of several operating-system concepts in one shell.

Dynamic input handling supports commands of varying lengths. Pipes and redirection allow commands to exchange data and work with files. Modular source files make the implementation easier to maintain and extend.

The monitoring thread demonstrates a basic use of multithreading, while debugging tools support more responsible software development.

From a sustainability perspective, the project emphasizes reusable code, maintainability, and low-resource command-line operat not measured any environmental savings, so we present sustainability as a design consideration rather than a measured result.

Made with GAMMA

Utility to Society and Future Scope

Utility

- Supports operating-systems education
- Demonstrates process management and inter-process communication
- Helps students practise C and POSIX APIs
- Builds debugging and testing skills

Limitations and proposed future work

- No actual Docker/OCI container lifecycle management
- Pipeline support is limited to two commands
- Improve monitoring-thread shutdown and cleanup
- Expand automated tests
- Add command history and better error reporting
- Consider future integration with a container runtime

Items in the right column are proposed future features, not current claims.

Made with GAMMA

Conclusion and Review Summary

- Modular shell implemented in C on Ubuntu
- Built-ins and external commands tested
- Two-command pipeline demonstrated
- Input and output redirection demonstrated
- Valgrind warning identified for further investigation
- AddressSanitizer test completed without visible sanitizer errors

Made with GAMMA